VIETNAM NATIONAL UNIVERSITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

- FINAL PROJECT REPORT -

Book Recommendation System: Design, Machine Learning Pipeline, and Evaluation

Council : Computer Science
University : Ho Chi Minh University of Technology
Faculty : Computer Science and Engineering
Instructor : Prof. Quản Thành Thơ
Author : Đặng Thế Đông A (2570791)

Ho Chi Minh City, May 2026

Declaration of Authenticity

I declare that this report is my own work, conducted under the supervision of my instructor. The results and discussions presented in this report are original to this project context and have not been published previously in this form.

All materials, methods, and references adopted from prior works are properly cited in the References section. Any use of external ideas, datasets, and technical foundations is acknowledged in accordance with academic integrity requirements.

Acknowledgements

I would like to express my sincere gratitude to my instructor and the members of the research group for their guidance, support, and valuable feedback throughout this project.

I also thank my family and friends for their continuous encouragement during the implementation and writing process of this report.

Abstract

This report documents a full-stack book recommendation system that combines a React web frontend, a FastAPI backend, and a collaborative filtering model based on matrix factorization (implemented via stochastic gradient descent with latent factors, commonly referred to as SVD-style factorization). The system serves three stakeholder roles—end users, administrators, and data scientists—and is trained on the public Book-Crossing dataset.

After preprocessing explicit ratings and training with 80 latent factors, the model achieves a test RMSE of **0.857** on held-out ratings, with binary classification metrics (threshold 3.5 stars) of **77.1%** accuracy, **78.3%** precision, **96.8%** recall, and **86.6%** F1-score. The report covers stakeholder benefits, algorithm theory, system architecture, data preprocessing, the AI training and deployment pipeline, and evaluation methodology.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

This chapter introduces the book recommendation problem, states the project objectives, defines scope and limitations, and outlines the structure of the remainder of this report.

Personalized book recommendation helps readers discover relevant titles in large catalogs where manual browsing is inefficient. This project implements a production-style web application that ingests the Book-Crossing dataset, exposes REST APIs for catalog and user interactions, and deploys a trained collaborative filtering model to rank books for authenticated users.

1.1 Objectives

The project goals are:

- Build a role-based web application (user, admin, data scientist).
- Train and evaluate an SVD-style matrix factorization model on real rating data.
- Persist model artifacts and serve personalized recommendations via API.
- Provide dashboards for catalog management, analytics, and model experimentation.

1.2 Scope and Limitations

The system uses the Book-Crossing CSV dumps for seeding and offline training. Inference relies on an *item-centric* saved model (book biases and factors) with online estimation of user profiles from a small number of in-app ratings. Users without ratings receive fallback recommendations based on genre preferences or popularity. Analytics KPIs on the data scientist dashboard include seeded demonstration data for engagement trends.

1.3 Report Structure

The remainder of this report is organized as follows:

- Chapter 2 identifies stakeholders and the benefits they derive from the system.
- Chapter 3 presents the theoretical background of recommendation algorithms used in this project.
- Chapter 4 describes the system design, including backend, frontend, and deployment flow.
- Chapter 5 documents the dataset and preprocessing steps.
- Chapter 6 details the AI pipeline for training and deployment.
- Chapter 7 reports evaluation results with quantitative metrics.
- Chapter 8 summarizes the user interface and concludes with future work directions.

Chapter 2

Stakeholder Analysis

This chapter identifies the stakeholders involved in the project and explains the potential benefits each group can derive from the book recommendation system.

2.1 Stakeholder Identification

Table 2.1 summarizes the primary stakeholders, their roles in the system, and the interfaces they use.

Table 2.1: Project stakeholders and system touchpoints

Stakeholder	**Role**	**Touchpoints**
End users (readers)	Browse, search, rate books; manage reading list; receive recommendations	Home (`/`), Search, Book Detail, Profile
Administrators	Manage book catalog; view top-rated analytics	Admin dashboard (`/admin`), `/api/v1/admin/*`
Data scientists	Train/tune models; monitor metrics and experiments	Data Scientist dashboard (`/data-scientist`), `/api/v1/ml/*`
Developers / maintainers	Implement APIs, database, ML pipeline	`backend/app/`, `Bookrecommendationsystemui/`
Course evaluators / institution	Assess design, methodology, and results	This report, live demonstration

2.2 Benefits by Stakeholder

End users benefit from personalized recommendations displayed on the home page, including a human-readable `reason` and a `predicted_rating` score. Rating books improves future recommendations through online user profile estimation. Search and filters reduce discovery time; the reading list tracks planned, active, and completed books.

Administrators maintain catalog quality via create/update/delete book operations and can monitor top-rated titles aggregated from the dataset. This supports content curation for a library or retail scenario.

Data scientists gain a reproducible training workflow: configurable hyperparameters, optional grid search over tuning spaces, persisted experiment history (`ModelExperiment`), and aggregate metrics (`ModelMetrics`). Training jobs are tracked in `ModelJob` with statuses `queued`, `running`, `completed`, and `failed`.

Developers inherit a modular FastAPI architecture with OpenAPI documentation, JWT authentication, and clear separation between API routers, database models, and ML code (`app/ml/svd.py`).

2.3 Representative Use Cases

1. **UC1 — Get recommendations:** User logs in → `GET /users/me/recommendations` → SVD scores unrated books or fallback applies.
2. **UC2 — Rate a book:** User submits stars → `PUT /users/me/ratings/{book_id}` → subsequent recommendations use updated profile.
3. **UC3 — Admin catalog:** Admin creates or edits a book → `POST/PATCH /admin/books`.
4. **UC4 — Train model:** Data scientist triggers `POST /ml/train` with hyperparameters → artifact saved to `dataset/svd_model_latest`.

Chapter 3

Theoretical Background: Recommendation Algorithms

This chapter presents the theory of the algorithms used in this project, including collaborative filtering, matrix factorization, evaluation metrics, and the fallback recommendation strategy.

3.1 Recommender System Taxonomy

Recommender systems predict user preference for items. Common approaches include:

- **Content-based filtering** — recommends items similar to those the user liked, using item features (genre, author, etc.).
- **Collaborative filtering** — exploits patterns across users and items from interaction data (ratings), without requiring rich item metadata.
- **Hybrid methods** — combine both signals; this project uses collaborative filtering as primary with a content/preference fallback.

3.2 Collaborative Filtering and the Rating Matrix

Let R denote a sparse user–item rating matrix where r_{ui} is the rating user u gave item i. Most entries are missing. The goal is to estimate $\hat{r}_{ui}$ for unobserved pairs and rank items by predicted score.

3.3 Matrix Factorization (SVD-Style Model)

Although the implementation is labeled "SVD," training uses **latent factor matrix factorization** optimized with stochastic gradient descent (SGD), not a full singular value

decomposition of R. The prediction function is:

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i \tag{3.1}$$

where μ is the global mean rating, b_u and b_i are user and item bias terms, and $\mathbf{p}_u, \mathbf{q}_i \in \mathbb{R}^k$ are k-dimensional latent factor vectors (hyperparameter `factors`).

For each observed rating (u, i, r_{ui}), the prediction error is $e_{ui} = r_{ui} - \hat{r}_{ui}$. SGD updates biases and factors to minimize squared error with L2 regularization λ:

$$b_u \leftarrow b_u + \eta \left(e_{ui} - \lambda b_u\right) \tag{3.2}$$

$$b_i \leftarrow b_i + \eta \left(e_{ui} - \lambda b_i\right) \tag{3.3}$$

$$\mathbf{p}_u \leftarrow \mathbf{p}_u + \eta \left(e_{ui}\mathbf{q}_i - \lambda \mathbf{p}_u\right) \tag{3.4}$$

$$\mathbf{q}_i \leftarrow \mathbf{q}_i + \eta \left(e_{ui}\mathbf{p}_u - \lambda \mathbf{q}_i\right) \tag{3.5}$$

where η is the learning rate (`learning_rate`) and λ is `regularization`. Predictions are clipped to $[0, 5]$.

Intuitively, latent factors capture hidden dimensions (e.g., literary vs. popular style, technical depth) that explain co-rating patterns across users and books.

3.4 Online User Profile Estimation

The deployed artifact stores *item* biases and factors plus a global mean. At inference time, a new or returning application user may have only a few ratings linked by ISBN. The system runs additional SGD steps (`estimate_user_profile`) to fit a temporary user bias and factor vector from those ratings, then scores candidate books with Equation (3.1).

3.5 Evaluation Metrics

RMSE (root mean squared error) measures regression quality on held-out ratings:

$$\text{RMSE} = \sqrt{\frac{1}{|T|} \sum_{(u,i,r) \in T} (r - \hat{r}_{ui})^2} \tag{3.6}$$

For classification-style metrics, ratings are binarized at threshold $\tau = 3.5$: a book is

"liked" if $r \geq \tau$. Predictions are positive if $\hat{r}_{ui} \geq \tau$. Standard definitions apply:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{3.7}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \tag{3.8}$$

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3.9}$$

High recall with moderate precision (as observed in our experiments) indicates the model tends to predict "liked" broadly, which suits discovery-oriented recommendation.

3.6 Fallback Strategy

If no trained model exists, the user has no ratings, or ISBN mapping fails, the API falls back to (1) pre-seeded `Recommendation` rows, or (2) genre-preference and popularity sorting from the `Book` table.

Chapter 4

System Design

This chapter describes the system design, including high-level architecture, backend and frontend components, database schema, recommendation serving flow, and technology stack.

4.1 High-Level Architecture

Figure 4.1 shows the three-tier architecture: React SPA, FastAPI backend, SQLite database, and filesystem model artifacts.

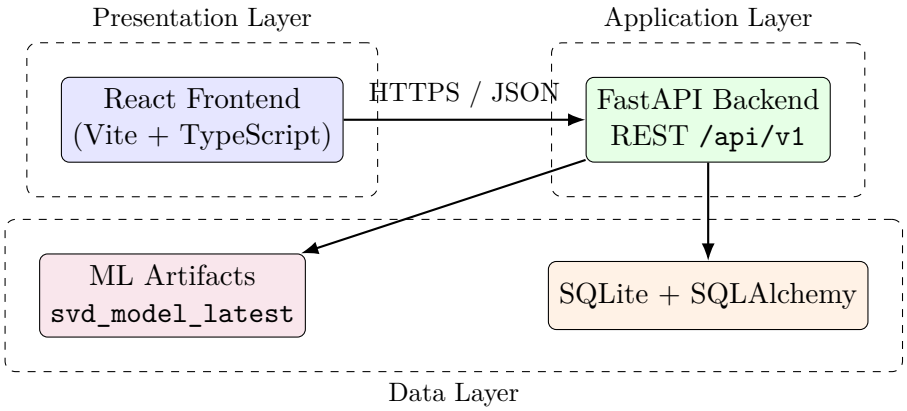

Figure 4.1: Three-tier system architecture for the book recommendation system.

4.2 Backend Design

The backend (`backend/app/`) is built with **FastAPI** and **SQLAlchemy**. On startup, tables are created and `seed_data()` loads books from `BX-Books.csv` if the database is empty.

API routers:

- `auth` — JWT login (`POST /auth/login`)

- `books` — paginated search, metadata, book detail
- `users` — profile and preferences
- `recommendations` — ratings, reading list, personalized recommendations
- `admin` — book CRUD (admin role only)
- `ml` — training, metrics, experiments (data scientist only)
- `analytics` — user activity, popular books, KPIs
- `health` — service health check

Role-based access control uses three roles defined in `RoleEnum`: `user`, `admin`, and `data_scientist`. Dependencies (`require_role`, `get_current_user`) enforce authorization on protected routes.

4.3 Database Schema

Core entities include:

- `User` — credentials, role, genre preferences
- `Book` — title, author, genre, ISBN, cover, aggregated rating
- `UserRating` — per-user star ratings (1–5)
- `ReadingListItem` — planned / reading / completed
- `Recommendation` — seeded or legacy recommendation rows
- `ModelExperiment`, `ModelMetrics`, `ModelJob` — ML lifecycle
- `AnalyticsUserActivity`, `AnalyticsPopularBook`, `AnalyticsTopRatedBook`

4.4 Frontend Design

The frontend (`Bookrecommendationsystemui/`) uses **React 18**, **Vite**, **React Router**, and **TanStack Query**. A shared `Layout` provides sidebar navigation and a role switcher for demo accounts.

API base URL defaults to `http://127.0.0.1:8000/api/v1`, configurable via `VITE_API_BASE_URL`.

Table 4.1: Frontend routes and purpose

Route	Component	Purpose
`/`	Home	Personalized recommendations grid
`/search`	Search	Filtered book search
`/book/:id`	BookDetail	Book metadata and actions
`/profile`	Profile	Reading history and preferences
`/admin`	AdminDashboard	Catalog and analytics (admin)
`/data-scientist`	DataScientistDashboard	ML training and metrics

4.5 Recommendation Serving Flow

1. Client requests `GET /users/me/recommendations?limit=6` with JWT.
2. Backend loads `svd_model_latest` from disk.
3. User ratings are joined to books by `book_id` $\rightarrow$ ISBN.
4. `estimate_user_profile` fits user bias and factors.
5. Up to 5000 unrated candidate books are scored; top-N returned with reason text.
6. If SVD path yields no items, seeded recommendations or preference/popularity fallback is used.

4.6 Technology Stack

Table 4.2: Technology stack

Layer	Technologies
Frontend	React, TypeScript, Vite, Tailwind CSS, shadcn/ui, Recharts
Backend	Python 3, FastAPI, SQLAlchemy, Pydantic, python-jose, passlib
ML	NumPy, custom SGD matrix factorization
Database	SQLite (via SQLAlchemy)
Dev tools	pytest, Uvicorn, Alembic

Chapter 5

Dataset Description and Preprocessing

This chapter describes the Book-Crossing dataset used in this project and the preprocessing steps applied for database seeding and model training.

5.1 Book-Crossing Dataset

The system uses the **Book-Crossing** dataset [1], a well-known benchmark for recommender systems research. Three CSV files are stored under `backend/dataset/`:

- `BX-Books.csv` — ISBN, title, author, year, publisher, cover image URLs
- `BX-Users.csv` — user demographics (available for extension; not used in current ML pipeline)
- `BX-Book-Ratings.csv` — User-ID, ISBN, Book-Rating

Files use semicolon delimiters and `latin-1` encoding. Raw explicit ratings are on a 1–10 scale; a value of **0** indicates implicit feedback (e.g., purchase) and is excluded from training.

Approximate file sizes in this deployment: `BX-Books.csv` ∼75 MB, `BX-Book-Ratings.csv` ∼30 MB.

5.2 Database Seeding Preprocessing

Function `seed_data()` in `app/db/seed.py` performs:

1. **Book import** — Parse `BX-Books.csv`; deduplicate by ISBN; cap at `SEED_BOOK_LIMIT` (default 20,000).
2. **Genre inference** — Keyword rules on title (e.g., mystery, romance, fantasy) assign a genre label.

3. **Rating aggregation** — Mean explicit rating per ISBN from `BX-Book-Ratings.csv`, converted to 0–5: $\min(5, \bar{r}/2)$.

4. **Demo data** — Seed users (reader, admin, data scientist), sample recommendations, reading list, and analytics rows.

5.3 ML Training Preprocessing

Function `load_ratings_from_csv` in `app/ml/svd.py`:

1. Parse ratings; skip invalid rows and implicit zeros.

2. Scale ratings: $r' = \min(5, r/2)$.

3. Filter users with fewer than `min_user_ratings` (default 2).

4. Filter books (ISBN) with fewer than `min_book_ratings` (default 2).

5. Random subsample to `max_ratings` (e.g., 30,000) with fixed `random_seed`.

6. Shuffle and split into train/test by `test_ratio` (default 0.2).

5.4 Data Quality Considerations

The Book-Crossing dataset is sparse and noisy: many users rate few books, and ISBNs may not overlap with the seeded application catalog. Inference maps application `Book.isbn` to model item indices; books outside the training vocabulary are skipped during scoring. Subsampling and filtering improve training stability at the cost of discarding long-tail interactions.

Chapter 6

AI Pipeline: Training and Deployment

This chapter details the AI pipeline for data training and deployment, from raw ratings CSV through model persistence to live recommendation serving.

6.1 Pipeline Overview

Figure 6.1 summarizes the end-to-end ML workflow from raw CSV to live recommendations.

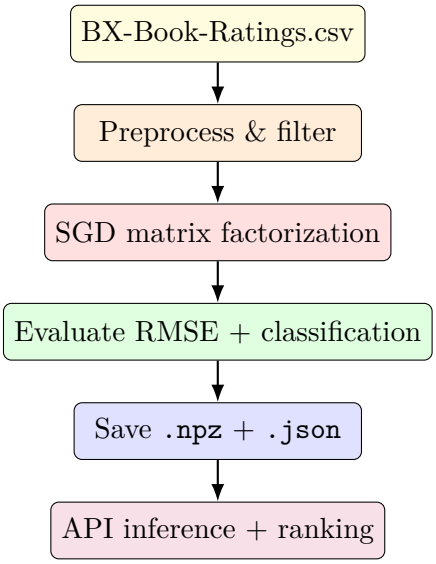

Figure 6.1: AI training and deployment pipeline.

6.2 Training API

Data scientists invoke `POST /api/v1/ml/train` with a JSON body specifying:

- Data config: `max_ratings`, `test_ratio`, `random_seed`, `min_user_ratings`, `min_book_ratings`
- Base hyperparameters: `factors`, `epochs`, `learning_rate`, `regularization`
- Optional tuning: `enable_tuning`, `tuning_space`, `max_trials`

The handler creates a `ModelJob`, loads ratings, trains the base configuration, optionally iterates a Cartesian product of tuning parameters (keeping the lowest test RMSE), upserts `ModelMetrics`, logs `ModelExperiment` rows, and writes the best model to `backend/dataset/svd`.

Minimum 1,000 ratings are required after filtering; otherwise training fails with HTTP 500.

6.3 Hyperparameter Tuning

When `enable_tuning` is true, the backend evaluates up to `max_trials` combinations from lists such as factors $\in \{50, 80, 120\}$, epochs $\in \{8, 10\}$, learning rates $\in \{0.005, 0.01\}$, and regularizations $\in \{0.02, 0.05\}$. Each trial is logged as an archived experiment; the best trial by test RMSE is marked active.

6.4 Model Artifact Format

`save_model_artifact` writes:

- `svd_model_latest.npz` — NumPy arrays `item_bias`, `item_factors`
- `svd_model_latest.json` — `global_mean`, `item_index` (ISBN $\rightarrow$ index)

User factors are *not* persisted; they are recomputed per request from the user's current ratings.

6.5 Deployment and Inference

At runtime, `recommendations.py` loads the artifact, builds the user profile, scores unrated catalog books, and returns the top-`limit` items with `predicted_rating` rounded to two decimals. Training is synchronous within the HTTP request (suitable for coursework/demo; production would use a background worker).

6.6 Seeded Demo Accounts

Password for all accounts: `password123`.

Table 6.1: Demo accounts for testing

Email	**Role**
`alex.johnson@example.com`	user
`admin@example.com`	admin
`datasci@example.com`	data_scientist

Chapter 7

Evaluation Results

This chapter reports evaluation results with clear metrics, experimental setup, interpretation, and qualitative assessment of recommendation quality.

7.1 Experimental Setup

Experiments were run using the implementation in `app/ml/svd.py` with the configuration in Table 7.1 (matching the project README quick-start defaults).

Table 7.1: Training configuration for reported results

Parameter	**Value**
Dataset	`BX-Book-Ratings.csv`
`max_ratings`	30,000
`test_ratio`	0.2 (6,000 test / 24,000 train)
`random_seed`	42
`min_user_ratings` / `min_book_ratings`	2 / 2
`factors`	80
`epochs`	10
`learning_rate`	0.01
`regularization`	0.05
Binary threshold τ	3.5

7.2 Quantitative Results

Table 7.2 reports metrics produced by `train_and_evaluate_svd` on the held-out test set. Training completed in approximately **1.5 seconds** for this configuration.

Table 7.2: Model evaluation metrics (single training run, May 2026)

Metric	Value
Train RMSE	0.622
Test RMSE	0.857
Accuracy (%)	77.05
Precision (%)	78.31
Recall (%)	96.77
F1-score (%)	86.57
Training duration (s)	1.52
Ratings used	30,000
Distinct users (train index)	10,688
Distinct books / ISBNs (train index)	15,674

7.3 Interpretation

RMSE: Test RMSE (0.857) exceeds train RMSE (0.622), indicating mild overfitting—expected with limited epochs and high model capacity relative to noisy data. Values below 1.0 on a 0–5 scale represent reasonable rating prediction for exploratory recommendation.

Classification metrics: High recall (96.8%) means the model rarely misses books a user would like (at $\tau = 3.5$). Precision (78.3%) is lower, reflecting false positives where the model predicts "like" for books the user rated below threshold. F1 (86.6%) balances these effects. Accuracy (77.1%) is consistent with class imbalance toward positive labels in the test split.

Practical impact: For the home page, ranking by predicted score prioritizes books likely to receive high stars, supporting discovery even when absolute rating error remains nonzero.

7.4 Hyperparameter Sensitivity

Further trials via `enable_tuning` can compare factor counts (50–120), learning rates, and regularization. Lower test RMSE generally correlates with better ranking quality; the data scientist dashboard stores per-trial RMSE in `ModelExperiment` for comparison.

7.5 Qualitative Assessment

For user `alex.johnson@example.com` with seeded ratings on three books, the API returns SVD-based recommendations with reason *"Predicted by SVD collaborative filtering from your ratings"* when the artifact exists and ISBNs align. Users without ratings see preference-based or popularity fallbacks, demonstrating graceful degradation.

Chapter 8

User Interface and Conclusion

This chapter summarizes the user interface surfaces and concludes the report with key achievements and future work directions.

8.1 User Interface Summary

The Figma-derived UI implements a consistent card-based design (purple accents, rounded corners). Key surfaces include:

- **Home** — 3-column recommendation grid with cover, rating, genre, and explanation panel.
- **Search** — genre and minimum-rating filters with paginated results.
- **Book Detail** — metadata, star display, reading-list actions.
- **Profile** — avatar, preferences, reading history.
- **Admin Dashboard** — book table with CRUD modals; top-rated chart.
- **Data Scientist Dashboard** — model overview, metrics cards (RMSE, accuracy, precision, recall, F1), experiment table, and training dialog wired to `POST /ml/train`.

8.2 Conclusion

This project delivers an end-to-end book recommendation system with clear stakeholder value: readers receive personalized suggestions, administrators manage catalog data, and data scientists control model training and evaluation. The theoretical foundation rests on collaborative filtering via matrix factorization; the implementation trains efficiently on Book-Crossing data and deploys a compact artifact for real-time inference with online user profiling.

Reported test metrics (RMSE 0.857, F1 86.6%) demonstrate that the pipeline produces a functional model suitable for demonstration and coursework evaluation.

8.3 Future Work

- Persist full user factors or use two-tower retrieval for faster candidate generation.
- Incorporate content features (author, genre embeddings) for hybrid recommendations and cold-start books.
- Replace synchronous training with job queues (Celery/RQ) and model versioning.
- Use production database (PostgreSQL) and containerized deployment.
- Conduct A/B tests on click-through and reading-list conversion.
- Expand evaluation with ranking metrics (NDCG@K, MAP@K).

Appendix A

API Endpoint Summary

Table A.1: Selected REST API endpoints

Method	**Path**	**Auth**
POST	`/auth/login`	Public
GET	`/books`	Public
GET	`/users/me/recommendations`	User
PUT	`/users/me/ratings/{book_id}`	User
GET/POST/PATCH/DELETE	`/admin/books`	Admin
POST	`/ml/train`	Data scientist
GET	`/ml/metrics`	Data scientist
GET	`/analytics/user-activity`	Data scientist

Appendix B

Sample Training Request

The following command illustrates how a data scientist triggers SVD training via the API (replace `<TOKEN>` with a valid JWT):

```
curl -X POST "http://127.0.0.1:8000/api/v1/ml/train" \
  -H "Authorization: Bearer <TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"max_ratings":30000,"test_ratio":0.2,
       "hyperparameters":{"factors":80,"epochs":10,
       "learning_rate":0.01,"regularization":0.05}}'
```

Appendix C

Source Code Layout

```
figma/
  backend/app/main.py       # FastAPI entry
  backend/app/ml/svd.py     # Training & inference
  backend/app/db/models.py  # SQLAlchemy models
  backend/dataset/          # BX CSV & model artifact
  Bookrecommendationsystemui/src/app/pages/
  report/report.tex         # This document
```

Bibliography

[1] Ziegler, C.-N., McNee, S. M., Konstan, J. A., and Lausen, G.: Improving Recommendation Lists Through Topic Diversification. In *Proceedings of the 14th International Conference on World Wide Web (WWW)*, 2005. (Book-Crossing dataset.)

[2] Koren, Y., Bell, R., and Volinsky, C.: Matrix Factorization Techniques for Recommender Systems. *Computer*, 42(8):30–37, 2009.

[3] Ramírez, S.: FastAPI Documentation. `https://fastapi.tiangolo.com/`, 2024.

[4] Meta Open Source: React Documentation. `https://react.dev/`, 2024.

[5] Book Recommendation Project: `backend/README.md` and `Bookrecommendationsystemui/README.md`. Repository documentation, 2026.